

ASSIGNMENT-1

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 1

Student Name: R.sandeep

Roll No.: A24126552111

Date: 15-09-2026

1. TITLE

Types of Inheritance in JavaScript Using ES6 Classes

2. AIM

To understand and implement the five types of inheritance in JavaScript — Single, Multilevel, Hierarchical, Multiple (via Mixins), and Hybrid — using JavaScript classes and inheritance mechanisms.

3. OBJECTIVE

- To understand the concept of inheritance and how it promotes code reuse in object-oriented programming.
- To use JavaScript class syntax to define base classes and derived classes using the extends keyword.
- To implement Single Inheritance where one derived class extends one base class.
- To implement Multilevel Inheritance where a chain of classes is formed.
- To implement Hierarchical Inheritance where multiple derived classes extend one base class.
- To simulate Multiple Inheritance using mixins.
- To implement Hybrid Inheritance by combining inheritance patterns in one program.

4. THEORY

Inheritance — a core principle of object-oriented programming that allows a class (derived/child) to acquire properties and methods of another class (base/parent), enabling code reuse and a logical hierarchy.

extends Keyword — used in a class declaration to establish inheritance between a derived class and its base class.

Method Overriding — a derived class can define its own method with the same name as a method in the base class.

Single Inheritance — one derived class inherits from one base class.

Multilevel Inheritance — inheritance occurs through a chain, such as Animal → Dog → Puppy.

Hierarchical Inheritance — two or more derived classes independently extend the same base class.

Multiple Inheritance (Mixin) — JavaScript does not support multiple class inheritance directly; mixins can combine additional behaviours into a class.

Hybrid Inheritance — a combination of two or more inheritance patterns in one program.

5. ALGORITHM / PROCEDURE

- Create separate JavaScript programs for Single, Multilevel, Hierarchical, Multiple, and Hybrid inheritance.
- Define a base class or object containing common behaviour.
- Create derived classes using extends where applicable.

- For multiple inheritance, combine behaviours using `Object.assign` and mixin objects.
- Create objects from the derived classes.
- Run each file in Node.js and verify the console output.

6. PROGRAM

Program 1: single_inheritance.js

```
class Animal {
  eat() {
    console.log("Animal eats food");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog barks");
  }
}

let dog = new Dog();

dog.eat();
dog.bark();
```

Program 2: Multilevel_Inheritance.js

```
class Animal {
  eat() {
    console.log("Animal eats food");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog barks");
  }
}

class Puppy extends Dog {
  play() {
    console.log("Puppy plays");
  }
}

class Cat extends Animal {
  meow() {
    console.log("Cat meows");
  }
}

let puppy = new Puppy();
let cat = new Cat();

puppy.eat();
puppy.bark();
puppy.play();

cat.eat();
cat.meow();
```

Program 3: Hierarchical_Inheritance.js

```
class Animal {
  eat() {
    console.log("Animal eats food");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog barks");
  }
}

class Cat extends Animal {
  meow() {
```

```

        console.log("Cat meows");
    }
}

let dog = new Dog();
let cat = new Cat();

dog.eat();
dog.bark();

cat.eat();
cat.meow();

```

Program 4: Multiple_Inheritance.js

```

class Animal {
    eat() {
        console.log("Animal eats");
    }
}

const Flying = {
    fly() {
        console.log("Animal can fly");
    }
};

const Swimming = {
    swim() {
        console.log("Animal can swim");
    }
};

class Duck extends Animal {}

Object.assign(Duck.prototype, Flying, Swimming);

let duck = new Duck();

duck.eat();
duck.fly();
duck.swim();

```

Program 5: Hybrid_Inheritance.js

```

class Animal {
    eat() {
        console.log("Animal eats");
    }
}

class Dog extends Animal {
    bark() {
        console.log("Dog barks");
    }
}

class Puppy extends Dog {
    play() {
        console.log("Puppy plays");
    }
}

class Cat extends Animal {
    meow() {
        console.log("Cat meows");
    }
}

let puppy = new Puppy();
let cat = new Cat();

puppy.eat();
puppy.bark();

```

```
puppy.play();
```

```
cat.eat();  
cat.meow();
```

7. CODE EXPLANATION

class Animal — defines the base class and its common eat() behaviour.

class Dog extends Animal — Dog inherits the eat() method from Animal and adds bark().

class Puppy extends Dog — Puppy inherits Animal and Dog behaviour and adds play(), demonstrating multilevel inheritance.

class Cat extends Animal — Cat independently inherits from Animal, demonstrating hierarchical inheritance.

Object.assign(Duck.prototype, Flying, Swimming) — copies methods from Flying and Swimming onto Duck, demonstrating a mixin-based multiple-behaviour approach.

new Dog(), new Cat(), new Puppy(), new Duck() — creates objects that can use inherited and their own methods.

8. TEST CASES AND EXECUTION DATA

TC-01	single_inheritance.js	dog.eat(); dog.bark();	Animal eats food; Dog barks	Pass
TC-02	Multilevel_Inheritance.js	puppy.eat(); puppy.bark(); puppy.play();	Animal eats food; Dog barks; Puppy plays	Pass
TC-03	Multilevel_Inheritance.js	cat.eat(); cat.meow();	Animal eats food; Cat meows	Pass
TC-04	Hierarchical_Inheritance.js	dog.eat(); dog.bark();	Animal eats food; Dog barks	Pass
TC-05	Hierarchical_Inheritance.js	cat.eat(); cat.meow();	Animal eats food; Cat meows	Pass
TC-06	Multiple_Inheritance.js	duck.eat(); duck.fly(); duck.swim();	Animal eats; Animal can fly; Animal can swim	Pass
TC-07	Hybrid_Inheritance.js	puppy.eat(); puppy.bark(); puppy.play();	Animal eats; Dog barks; Puppy plays	Pass
TC-08	Hybrid_Inheritance.js	cat.eat(); cat.meow();	Animal eats; Cat meows	Pass

9. OUTPUT

Output 1: Single Inheritance

Animal eats food
Dog barks

Output 2: Multilevel Inheritance

Animal eats food
Dog barks
Puppy plays

Output 3: Hierarchical Inheritance

Animal eats food
Dog barks
Animal eats food
Cat meows

Output 4: Multiple Inheritance

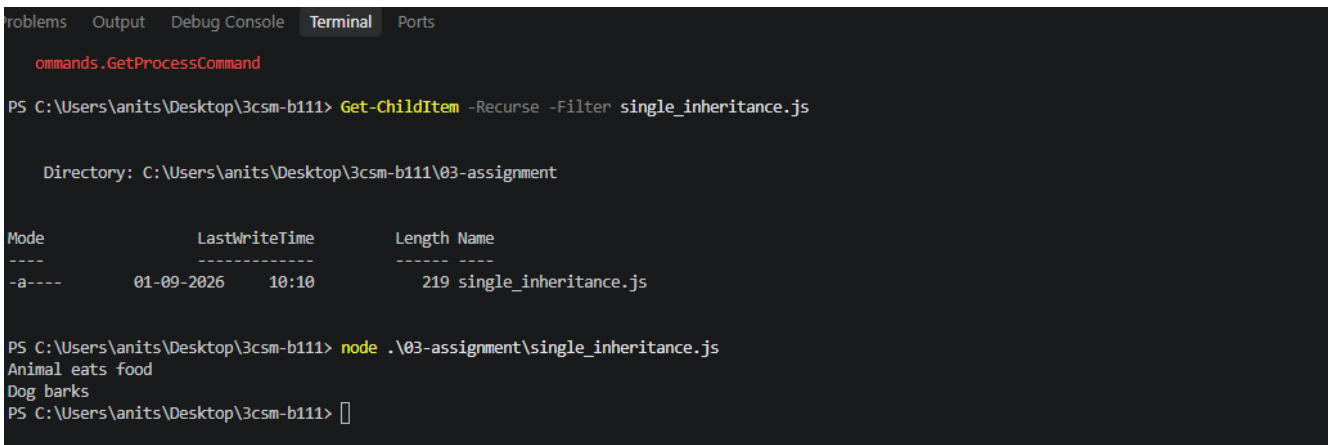
Animal eats
Animal can fly
Animal can swim

Output 5: Hybrid Inheritance

Animal eats
Dog barks
Puppy plays
Animal eats
Cat meows

ACTUAL NODE.JS EXECUTION OUTPUT

Output Screenshot 1 — single_inheritance.js



```
Problems Output Debug Console Terminal Ports
ommands.GetProcessCommand

PS C:\Users\anits\Desktop\3csm-b111> Get-ChildItem -Recurse -Filter single_inheritance.js

Directory: C:\Users\anits\Desktop\3csm-b111\03-assignment

Mode                LastWriteTime         Length Name
----                -
-a----             01-09-2026   10:10             219 single_inheritance.js

PS C:\Users\anits\Desktop\3csm-b111> node .\03-assignment\single_inheritance.js
Animal eats food
Dog barks
PS C:\Users\anits\Desktop\3csm-b111> 
```

Output Screenshot 2 — Multilevel_Inheritance.js

```
PS C:\Users\anits\Desktop\3csm-b111> node .\03-assignment\Multilevel_Inheritance.js
Animal eats food
Dog barks
Puppy plays
PS C:\Users\anits\Desktop\3csm-b111> 
fwd-i-search: _
```

Output Screenshot 3 — Hierarchical_Inheritance.js

```
PS C:\Users\anits\Desktop\3csm-b111> node .\03-assignment\Multilevel_Inheritance.js
Animal eats food
Dog barks
Puppy plays
PS C:\Users\anits\Desktop\3csm-b111> node .\03-assignment\Hierarchical_Inheritance.js
Animal eats food
Dog barks
Animal eats food
Cat meows
PS C:\Users\anits\Desktop\3csm-b111> █
```

Output Screenshot 4 — Multiple_Inheritance.js

```
PS C:\Users\anits\Desktop\3csm-b111> node .\03-assignment\Multiple_Inheritance.js
Animal eats
Animal can fly
Animal can swim
PS C:\Users\anits\Desktop\3csm-b111> █
```

```
PS C:\Users\anits\Desktop\3csm-b111> node .\03-assignment\Hybrid_Inheritance.js
Animal eats
Dog barks
Puppy plays
Animal eats
Cat meows
PS C:\Users\anits\Desktop\3csm-b111> 
```

10. RESULT

Thus, the five inheritance concepts — Single, Multilevel, Hierarchical, Multiple (via mixins), and Hybrid — were successfully demonstrated using the supplied JavaScript programs. The actual Node.js execution screenshots have also been included as evidence of the program outputs.